Software Requirements Specification

for

Online Voting System

Version 1.0 approved

Prepared by

Henil Gamit	**23CP058**	**Gamithenil384@gmail.com**
Yuvraj Malakiya	**23CP064**	**Yuvrajmalakiya2006@gmail.com**

Organization: BVM Engineering College

Instructor

Hemant Vasava

Vishal Patel

Course: Software Engineering (3CP07)

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 2
1.5 References 2
2. Overall Description 3
2.1 Product Perspective 3
2.2 Product Functions 3
2.3 User Classes and Characteristics 4
2.4 Operating Environment 4
2.5 Design and Implementation Constraints 4
2.6 User Documentation 4
2.7 Assumptions and Dependencies 5
3. External Interface Requirements 5
3.1 User Interfaces 5
3.2 Hardware Interfaces 6
3.3 Software Interfaces 6
3.4 Communications Interfaces 7
4. System Features 7
4.1 Admin Control & Institutional Setup 7
4.2 HOD Departmental Management 8
4.3 Teacher Class Management 9
4.4 Student Participation & Secure Voting 9
5. Other Nonfunctional Requirements 10
5.1 Performance Requirements 10
5.2 Safety Requirements 11
5.3 Security Requirements 11
5.4 Software Quality Attributes 12
5.5 Business Rules 12
6. Other Requirements 12
6.1 Database Requirements 12
6.2 Legal & Compliance Requirements 13
Appendix A: Glossary 13
Appendix B: Analysis Models 14
Appendix C: To Be Determined List 14

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

The purpose of this document is to provide a comprehensive Software Requirements Specification (SRS) for the **Online Voting System (OVS)**. This document defines the functional and non-functional requirements to ensure a secure, transparent, and hierarchical digital voting platform for educational institutions. It serves as a primary reference for the development team, project supervisors, and quality assurance testers.

1.2 Document Conventions

To ensure clarity throughout this document, the following role-based conventions are used:

- **Admin:** The high-level platform administrator responsible for system-wide configurations and college-level oversight.
- **HOD (Head of Department):** The department-level manager responsible for coordinating teachers and department-specific elections.
- **Teacher:** The class-level facilitator responsible for managing student rosters and class-level elections.
- **Student:** The end-user who participates in voting and can act as a candidate.
- **System:** The full-stack digital democracy platform encompassing the web interface and backend API.

1.3 Intended Audience and Reading Suggestions

This document is prepared for the following stakeholders:

- **Developers:** To guide the system implementation of the MERN stack and WebSocket integrations.
- **Testers:** To facilitate verification and validation using Jest and React Testing Library.

- **Project Supervisors:** To review project scope, security implementations, and functional alignment.
- **System Administrators:** To understand the deployment, environment configuration, and maintenance requirements.

Reading Suggestion: Readers should first review the **Overall Description** (Section 2) for context before moving to the **Detailed Functional Requirements** in the System Features section.

1.4 Product Scope

The **Online Voting System** is a secure, web-based platform designed to automate and digitize the election process in colleges. The scope of the project includes:

- **Multi-Role Hierarchy:** Secure management of Admins, HODs, Teachers, and Students.
- **Election Levels:** Capability to host College, Department, and Class-level elections.
- **Security Features:** Implementation of JWT-based authentication, account lockout mechanisms, and OTP-verified voting.
- **Real-time Interaction:** Live election updates and instant notifications using WebSockets.
- **Reporting:** Generation of audit trails and visual result statistics.

1.5 References

- **IEEE 830-1998:** Recommended Practice for Software Requirements Specifications.
- **OWASP Security Guidelines:** For input sanitization and injection protection standards.
- **Project Analysis Document:** Internal documentation for the OVS modern full-stack architecture.

2. Overall Description

2.1 Product Perspective

The Online Voting System (OVS) is an independent, full-stack web application built on the MERN (MongoDB, Express, React, Node.js) architecture. The system operates as a secure digital democracy platform that integrates with:

- **Email Notification Services:** Utilizing **Nodemailer** for OTP delivery and secure voting receipts.
- **Secure Authentication Services:** Implementing **JWT-based** role-based access control (RBAC) to manage hierarchical permissions.
- **Real-time Communication:** Leveraging **WebSockets** for instant election updates and live results.

2.2 Product Functions

The major system functions are categorized by the administrative hierarchy:

- **User Management:** Registration and management of HODs, Teachers, and Students across specific departments and classes.
- **Election Management:** Creation and configuration of **College, Department, and Class-level** elections.
- **Secure Voting:** Multi-stage voting process featuring candidate selection and **OTP verification**.
- **Real-time Notifications:** Notice creation and broadcasting targeted at specific user levels (System-wide, Department-wide, or Class-specific).
- **Analytics and Reporting:** Real-time result counting, visual data representation, and comprehensive **audit trails**.

2.3 User Classes and Characteristics

User Class	Description	Skill Level
Admin	Full system oversight; manages departments, HODs, and college-wide elections.	Advanced
HOD	Department-level manager; creates classes, registers teachers, and oversees department elections.	Intermediate
Teacher	Class-level facilitator; manages student rosters and conducts class-level elections.	Intermediate
Student	End-user; acts as a voter and/or candidate in eligible elections.	Basic

2.4 Operating Environment

- **Web Browsers:** Optimized for **Chrome, Firefox, Edge, and Safari**.
- **Responsiveness:** Fully mobile and desktop compatible via **Tailwind CSS v4**.
- **Server Environment:** Node.js 18+ runtime with **Redis** for session and OTP caching.
- **Database: MongoDB** for flexible, document-based storage of user and election data.

2.5 Design and Implementation Constraints

- **Security Standards:** Must include input sanitization (XSS protection), rate limiting, and Bcrypt password hashing.
- **Data Integrity:** Ensuring one-vote-per-student through strict database relations and session tracking.
- **Tech Stack:** Strict adherence to React 19.2.0 and Vite for the frontend build tool.

2.6 User Documentation

The system will be delivered with the following documentation:

- **Online Help:** Integrated tooltips and a "Help" section within each user dashboard.
- **Role-Specific User Manuals:** PDF guides tailored for Admin, HOD, Teacher, and Student workflows.
- **System Setup Guide:** Technical documentation for environment configuration (.env setup) and database seeding.

2.7 Assumptions and Dependencies

- **Assumptions:** It is assumed that students have access to a valid email address for receiving OTPs.
- **Dependencies:** The system depends on MongoDB Atlas (or a local instance) and a Redis server for caching. Third-party mail servers (e.g., Gmail or SendGrid) are required for Nodemailer functionality.

3. External Interface Requirements

3.1 User Interfaces

The system shall provide a modern, responsive web interface built with **React 19.2.0** and **Tailwind CSS v4**.

- **GUI Standards:** A card-based layout with modern gradient backgrounds and Lucide React icons.
- **Role-Based Dashboards:**
  - **Admin/HOD/Teacher:** Will feature a sidebar for navigation (User Management, Elections, Notices) and a statistical overview grid.
  - **Student:** Will feature a simplified "Ballot Box" view focusing on active elections and a "Notice Feed."
- **Standard Elements:** Every screen shall include a top navigation bar with a **Profile** dropdown, **Notifications** bell, and a **Logout** button.

- **Error Messaging:** Standardized "Toast" notifications (top-right corner) for success, error, and warning messages. Input fields must show red validation text for invalid data (e.g., "Invalid Email Format").
- **Components:** UI is required for Login/Registration, Election Creation, Candidate Management, OTP Verification Modal, and Results Visualization (Charts).

3.2 Hardware Interfaces

The OVS is a web-based application and does not require specialized custom hardware. It shall support:

- **Supported Device Types:** PC, Laptop, Tablets, and Smartphones.
- **Interaction:** Standard input via Keyboard, Mouse, and Touchscreen.
- **Nature of Interaction:** The software interacts with the hardware's network interface controller (NIC) to send/receive data via TCP/IP protocols.
- **Display:** Minimum resolution support of **320px** (mobile) up to **4K** (desktop) via responsive breakpoints.

3.3 Software Interfaces

The system relies on the following software integrations and libraries:

- **Database (MongoDB):** The primary data store. Connection is managed via **Mongoose ODM**. Data items include JSON-formatted User profiles, Election configurations, and Vote objects.
- **Caching (Redis):** Used specifically for high-speed retrieval of **OTP (One-Time Passwords)** and session tokens.
- **Authentication (JWT):** The interface between the Frontend and Backend for secure communication. Every API request must include a Bearer Token in the HTTP header.
- **Encryption (Bcryptjs):** A library interface used during the registration and login process to hash and compare passwords.
- **Email Service (Nodemailer):** Connects to an external SMTP server (e.g., Gmail, SendGrid) to dispatch election notices and OTPs to users.

3.4 Communications Interfaces

- **Network Protocols:** All communications between the client (Browser) and the server (Node.js) shall occur over **HTTPS** using **TLS 1.3** for encryption.
- **Real-time Communication: WebSockets (WS)** shall be used to push live election result updates and instant notices to active user sessions.
- **API Standards:** The backend shall expose a **RESTful API** using JSON for all request and response payloads.
- **Email Standards:** SMTP protocol for outgoing messages. Emails shall be formatted in HTML for professional presentation using institutional branding.
- **Rate Limiting:** Communication interfaces shall be protected by rate-limiting middleware to prevent Brute Force and DoS attacks.

4. System Features

4.1 Feature: Admin Control & Institutional Setup

4.1.1 Description and Priority

This feature allows the top-level Administrator to build the institutional structure by creating departments and classes and appointing heads.

- **Priority:** High (Critical for system initialization).

4.1.2 Stimulus/Response Sequences

- **Stimulus:** Admin selects "Create Department" and enters a name.
- **Response:** System validates the name, creates the record in MongoDB, and updates the Department list.
- **Stimulus:** Admin registers a new HOD and selects an assigned department.
- **Response:** System creates the HOD account and links their DepartmentID to the chosen department.

4.1.3 Functional Requirements

- **REQ-AD-1:** The system shall allow the Admin to create and delete Departments.
- **REQ-AD-2:** The system shall allow the Admin to register HODs and assign them to a specific Department.
- **REQ-AD-3:** The system shall allow the Admin to create Classes for any specific Department.
- **REQ-AD-4:** The system shall allow the Admin to create College-level elections and view global results.
- **REQ-AD-5:** The system shall allow the Admin to broadcast notices to all users.

4.2 Feature: HOD Departmental Management

4.2.1 Description and Priority

Provides the Head of Department with tools to manage their specific department, including teacher assignments and department-level elections.

- **Priority:** High.

4.2.2 Stimulus/Response Sequences

- **Stimulus:** HOD navigates to "Manage Teachers" and enters teacher details.
- **Response:** System creates the Teacher account and restricts their scope to the HOD's department.
- **Stimulus:** HOD creates a Department-level election.
- **Response:** System flags the election so only students within that DepartmentID can participate.

4.2.3 Functional Requirements

- **REQ-HOD-1:** The system shall restrict HOD actions to only their assigned department.
- **REQ-HOD-2:** The system shall allow HODs to create/manage classes and register/remove Teachers.
- **REQ-HOD-3:** The system shall allow HODs to register/remove Students within their department.

- **REQ-HOD-4:** The system shall allow HODs to create Department-level elections and view results.
- **REQ-HOD-5:** The system shall allow HODs to post notices visible to their department's teachers and students.

4.3 Feature: Teacher Class Facilitation

4.3.1 Description and Priority

Enables Teachers to manage the specific student rosters they are assigned to and conduct localized class elections.

- **Priority:** Medium.

4.3.2 Stimulus/Response Sequences

- **Stimulus:** Teacher clicks "Register Student" and provides student details.
- **Response:** System adds the student and links them to the Teacher's ClassID.
- **Stimulus:** Teacher initiates a Class Election.
- **Response:** The system enables the voting portal only for students belonging to that specific class.

4.3.3 Functional Requirements

- **REQ-TC-1:** The system shall allow Teachers to register and remove students from their assigned class.
- **REQ-TC-2:** The system shall allow Teachers to create Class-level elections.
- **REQ-TC-3:** The system shall display election results only to the students and the teacher of that class.
- **REQ-TC-4:** The system shall allow Teachers to post class-specific notices.

4.4 Feature: Student Participation & Secure Voting

4.4.1 Description and Priority

The core end-user feature allowing students to vote in eligible elections and view institutional updates.

- **Priority:** High (Core Value).

4.4.2 Stimulus/Response Sequences

- **Stimulus:** Student selects a candidate and clicks "Vote."
- **Response:** System generates an OTP, sends it to the student's email, and opens the verification modal.
- **Stimulus:** Student enters the correct OTP.
- **Response:** System records the vote, increments the candidate's count, and marks the student as "Voted" to prevent duplicates.

4.4.3 Functional Requirements

- **REQ-ST-1:** The system shall show students notices from all three levels (Admin, HOD, Teacher).
- **REQ-ST-2:** The system shall require Email OTP verification for every vote cast.
- **REQ-ST-3:** The system shall allow students to view results for elections at their specific level.
- **REQ-ST-4:** The system shall restrict a student's voting eligibility based on their DepartmentID and ClassID.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- **Response Time:** The system shall respond to standard HTTP requests (e.g., loading the dashboard) within **2 seconds** under normal load.
- **Real-time Updates:** Election results broadcasted via WebSockets must update on all connected client screens within **500ms** of a vote being successfully recorded.

- **Concurrent Users:** The system must support at least **500 concurrent users** voting simultaneously without a degradation in performance, specifically using **Redis** to handle high-frequency OTP sessions.
- **Throughput:** The backend must be capable of processing at least **50 votes per second** to ensure no bottlenecks during peak election hours.

5.2 Safety Requirements

- **Data Integrity:** In the event of a server crash or database disconnection, the system must ensure no "partial votes" are recorded. The voting process must be **atomic** (all-or-nothing).
- **Session Safety:** If a student's session expires or they lose connection mid-vote, the system must allow them to restart the process without marking them as "Already Voted" until the final confirmation is sent.
- **Backup:** Daily automated backups of the MongoDB database must be performed to prevent data loss due to hardware failure.

5.3 Security Requirements

- **Authentication:** Access is strictly controlled via **JWT (JSON Web Tokens)** stored securely in the client-side state. Tokens must expire after 2 hours of inactivity.
- **Voter Verification:** Every vote must be authenticated by a **6-digit OTP** sent to the student's registered email address.
- **Data Protection:** All passwords must be hashed using the **Bcrypt** algorithm with a salt factor of 10. No plain-text passwords shall ever be stored.
- **Authorization (RBAC):** The system must implement strict **Role-Based Access Control** middleware to ensure, for example, a Teacher cannot access HOD-level department management routes.
- **Input Sanitization:** All user inputs must be sanitized using libraries like DOMPurify (frontend) and express-validator (backend) to prevent **XSS (Cross-Site Scripting)** and **NoSQL Injection**.

5.4 Software Quality Attributes

- **Availability:** The system should maintain **99.5% uptime** during active election periods.
- **Usability:** The interface must be intuitive enough that a **Student (Basic skill level)** can cast a vote within 3 clicks after logging in, without prior training.
- **Maintainability:** The codebase must follow a modular structure (Controllers, Models, Routes) to allow developers to update specific features (like adding a new election level) without affecting the core voting logic.
- **Correctness:** The voting tallies must be **100% accurate**. The sum of all individual votes for an election must always equal the total "Vote Count" displayed in the results.

5.5 Business Rules

 Candidate Eligibility: Only users registered with the "Student" role can be added as candidates for any election level.

 One-Vote Rule: A student is permitted to vote only once per specific Election ID. The system must check the Vote model for existing entries before allowing a new submission.

 Hierarchy Constraint: An HOD can only manage the department they are assigned to; they cannot view or edit data from other departments.

 Election Level Logic:

- **College Elections:** All students in the database are eligible.
- **Department Elections:** Only students with a matching DepartmentID are eligible.
- **Class Elections:** Only students with a matching ClassID are eligible.

 Notice Visibility: Notices created by a Teacher must only be visible to students within that Teacher's specific class.

6. Other Requirements

6.1 Database Requirements

The system shall utilize **MongoDB** as the primary data store. The database must:

- Support **Mongoose ODM** for schema validation.
- Store unique indexes for user emails and voting records to ensure data integrity.
- Maintain a separate **Redis** instance for storing volatile data like 6-digit OTPs with an expiration (TTL) of 5–10 minutes.

6.2 Legal & Compliance Requirements

- **Data Privacy:** The system must comply with institutional data privacy policies, ensuring student information is not shared with third parties.
- **Anonymity:** While the system tracks who has voted (to prevent duplicates), the link between a specific user and their specific choice must be encrypted or separated to maintain ballot secrecy.

Appendix A: Glossary

- **Admin:** The highest authority; manages the college-level structure.
- **HOD:** Head of Department; manages departmental faculty and students.
- **Teacher:** Class-level coordinator; manages specific student groups.
- **Student:** The end-user who can vote or stand as a candidate.
- **OTP (One-Time Password):** A unique 6-digit code sent via email to verify a voter's identity.
- **JWT (JSON Web Token):** A secure method for transmitting information between the server and the client as a JSON object.
- **MERN Stack:** A collection of technologies (MongoDB, Express, React, Node.js) used to build the application.
- **RBAC (Role-Based Access Control):** A method of restricting system access to authorized users based on their specific role.

Appendix B: Analysis Models

B.1 Entity Relationship Diagram (ERD)

The system logic is based on the following relational structure:

- **Department** → contains many **Classes**.
- **Class** → contains many **Students** and one **Teacher**.
- **Department** → overseen by one **HOD**.
- **Election** → linked to a **Level** (College, Dept, or Class).
- **Vote** → links a **User** to a **Candidate** within a specific **Election**.

B.2 Data Flow Diagram (DFD)

The flow of information follows a strict top-down authorization:

1. **Admin** creates the "Department" and "HOD" entities.
2. **HOD** populates the "Class" and "Teacher" entities.
3. **Teacher** manages the "Student" entity.
4. **Student** generates "Vote" data which flows back into the "Results" analytics seen by all higher roles.

Appendix C: Issues List

Note: no issues yet